Software Requirements Specification

for

Pantry Inventory Management System

Version 1.8

Prepared by Andrew Feran, Arselle Conteh, Benjamin Bruinsma, Davi Borges, Nickolas Diaz

Florida Polytechnic University

06/01/2025

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction
- 1.1 Purpose
- 1.2 Intended Audience and Reading Suggestions
- 1.3 Product Scope

2. Overall Description
- 2.1 Product Perspective
- 2.2 Product Functions
- 2.3 User Classes and Characteristics
- 2.4 Operating Environment
- 2.5 Design and Implementation Constraints
- 2.6 User Documentation
- 2.7 Assumptions and Dependencies

3. External Interface Requirements
- 3.1 Hardware Interfaces
- 3.2 Software Interfaces
- 3.3 Communications Interfaces

4. System Features
- 4.1 Private and Protected Inventory
- 4.2 Item Management System
- 4.3 Pantry Search
- 4.4 Shopping List Creation
- 4.5 Remainder of Expiration Date
- 4.6 Basic Security Measures

5. Other Nonfunctional Requirements
- 5.1 Performance Requirements
- 5.2 Safety Requirements
- 5.3 Security Requirements
- 5.4 Software Quality Attributes

6. Other Requirements
- 6.1 Database Requirements

7. Use Case Model
- 7.1 Use Case Diagram
- 7.2 Use Case Descriptions

System Terms:

Revision History

Name	Date	Reason For Changes	Version
	5/30/25	Init. Ver.	1.0
Andrew Feran	5/31/25	Completed Sec 5. + 6.1	1.1
Nickolas Diaz	5/31/25	Completed Sec. 1, + 2.2 added to 5.3	1.2
Arselle Conteh	5/31/25	Completed Sec. 3, +2.4	1.3
Nickolas Diaz	6/1/25	Completed section 2.1 and 2.5	1.4
Davi Borges	6/1/25	Completed Sec 4.	1.5
Davi Borges	6/1/25	Adjusted page numbering and formatting.	1.6
Andrew Feran	6/1/25	Small tweaks + Glossary	1.7
Andrew Feran	6/1/25	Section 7	1.8

1. Introduction

1.1 Purpose

The pantry management system is a web app that allows users to manage their own pantry systems, which includes adding, modifying and deleting items as well as adding expiration dates. Users can also create shopping lists for missing items and have remainders for their products. This app is meant for users/businesses to manage their own businesses and be able to keep track of all their items.

1.2 Intended Audience and Reading Suggestions

Developers: sections 1-7 Know the application to the front and back and develop new features and maintain the software.
Marketing staff: sections 1,2,4 Know what the product is and market the features
Users: sections 1,2 Be able to use the software
Testers: sections 1,2,4,5 Know what the product features are and how to test and break it.
Documentation writers: sections 1,2 Write documentation for all the different types of readers
Stockholders/CEO: 1-4 Know the application and be able to manage the business
Project managers: sections 1-7 know the whole project and be able to manage devs to develop and maintain, be able to explain the team's work to the Stockholders/CEO. Make the marketing staff market the material. Make testers test the software and manage the documentation of it.

1.3 Product Scope

The purpose of the pantry inventory management system is to make it easier for businesses and employees to manage their pantry. Employees will be able to better manage their items and users will be able to search and purchase items more easily. The goal of this business is to be able to provide as much help to users regarding managing their pantry.

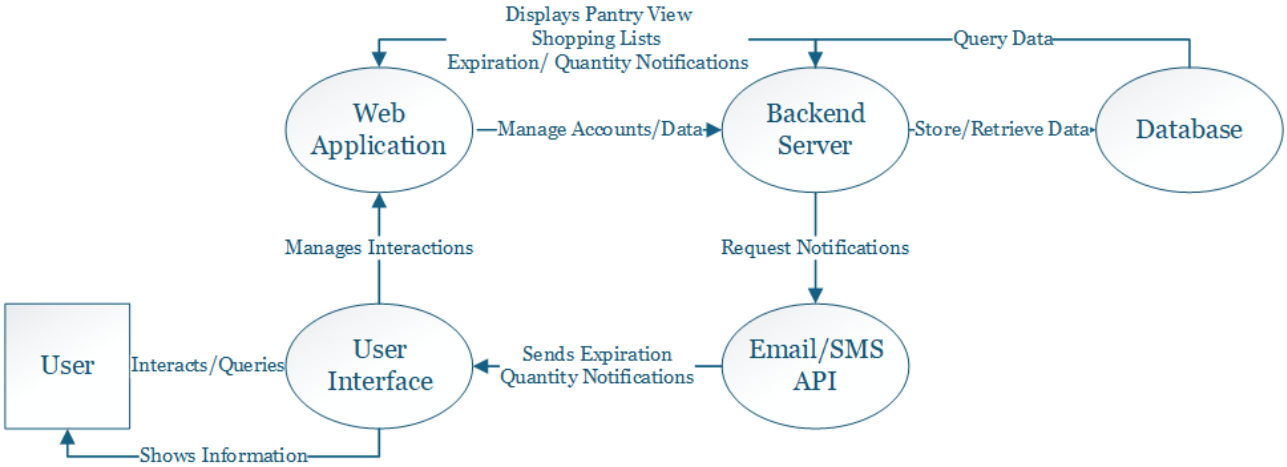
2. Overall Description

2.1 Product Perspective

The pantry inventory management system is a new self-contained product. It is meant to solve the problem of businesses/households keeping track of their pantry. It is meant to help people manage their inventory efficiently and cleanly and to keep track/notify the quantity of items and expiration dates. The product is aimed for everyone of all sizes, it is simple enough for families and full of features for large businesses.

Components of the system:
user interface through the web
web application
backend server
database
user/manager
notification (in-app, or SMS, email)
Third party services/api

Diagram:



2.2 Product Functions

2.2.1 Core features

Private and protected inventory: by providing user authentication to the staff and users
Item management system: for pantry items which allows users to add, update, and remove items with information such as names, quantities, and expiration dates
Pantry search: allows users to search items according to quantity, name, price and expiration date
Shopping list creation: users can create shopping lists, and buy items from that list
Remainders of expiration date: Users will be notified if their items are nearing their expiration date, possibly via push/email or highlighting.

2.2.2 Basic Security Measures

Input validation: All input into the server must be sanitised before being processed
Role-based access control: Reading and editing pantries are only given to the users
Session: Once logged in users must remain logged in during their session, using Cookies-Based Authentication, token authentication through (JWT).

2.3 User Classes and Characteristics

User Class	Frequency of Use	Technical Expertise	Notes
Regular User	Medium	Low	Class represents customers who use the product
System Admins	Medium	High	Maintain system consistency and usability
Developers and Testing	High	High	Use the system during testing and development

- **Regular Users:**
  - Description: Everyday users who use the application to manage their panty items
  - Use the system: Daily or Weekly
  - Importance: High, this system is designed to accommodate these users. This user's satisfaction is the most important user of the system.
  - Technical Expertise: Low, basic computer skills
- **System Admins:**
  - Description: The people in charge of managing the system. They are in charge of maintaining the system and troubleshooting system errors.
  - Use the system: As needed when issues surface
  - Importance: Medium, a degree of usability is required but unnecessary to place effort into
  - Technical Expertise: Advanced, experience in information technology is required
- **Developers:**
  - Description: The team of developers who build the system
  - Use the system: High during testing and development
  - Importance: Medium, only relevant during development
  - Technical Expertise: High, experience in software development

2.4 Operating Environment

The environment for this product is a web-based application that is designed to operate within a standard client-server architecture. The client side will need to be able to run more modern web browsers (such as Safari, Google Chrome, and Edge), and it needs to support HTML5, CSS3, and JavaScript (ES6). As for the server side, a Linux-based environment will likely need to be integrated with an HTTP server (something similar to Ubuntu Server 22.04 LTS/Apache HTTP Server 2.4) which will act as the host. We will use MySQL to run the database for this system due to its data managing capabilities. The system needs to be peacefully integrated with the email service, as well as the security software.

2.5 Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

Major constraint is finding a place to host the system whether it is a hosting service or personal hardware. We need to be able to deploy a python webserver, Mysql database and potential api's for the notifications.

Python web server: It needs to process data coming into and out of the database. It will need to manage all the functions for the entire system. It needs a secure connection to the MySQL database, any APIs, and to the client. It also has to manage users throughout their session for secure access.

MySQL database: The schema design needs to be able to store all the information needed, SQL queries to modify, add, delete and query the data. Data needs to be processed before entering the database. If the schema changes so does the way it is accessed. Performance tuning: tables need to be indexed and have good connections to other tables.

User interface: It has to be user friendly and show the necessary information, and actions through buttons written in HTML and CSS. It also has to support the logging and logging out of users.

Web interface: It has to connect the client interfaces to the web server to perform the actions of the users. It has to manage all the cookies used and it will be written in javascript.

API: It will need to be configured to send notifications to email addresses.

Security: The web app needs secure password storage and session management, it needs good role access control for the lists and inventory and it needs good api security

Mitigating Errors: error responses should not be sent back to the client. Errors needs to be wrapped within try catch blocks so if one happens the web app won't break

Documentation: If a error or a new feature is needed modification to the documentation is needed

2.6 User Documentation

- User manuals: PDF documentation
    - An in-depth analysis of the system functions and how to use them
    - Specific help can be found in this document for most problems
- On-line help:
    - A small button marked '?' with specific help when the user is stuck

    - A FAQ page of issues users have and solutions to their problem
- Tutorials: Quick-start tutorial
    - A quick presentation of mainly images showing the basic functions the user can use in the system
    - A short video showcasing the various functionality of the application

2.7 Assumptions and Dependencies

We assume that:

- There are no more than 500 items in the pantry at a time
- There are no more than 10 concurrent users
- The HTTPS protocol is secure
- The probability of collision of hashed passwords is zero due to cryptographic primitives
- Users may access this site from any number of web supported device types

This system will inherently rely on the security and reliability of:

- Python 3 as the codebase
- Flask as the backend of the web application
- MySQL as the database
- Apache2 as the web server
- Ubuntu as the host OS

3. External Interface Requirements

3.1 Hardware Interfaces

Some of the physical characteristics of this interface consist of the User Devices, such as the physical PC, tablet, smart phones, or anywhere that this interface is being accessed from. The backend server (the web server) is one of the main logical characteristics of the interface and its main purpose would be handling requests or processing data. Other logical characteristics would consist of the database (that takes care of the items and queries) as well as a potential notification system that keeps all of the users aware of all the changes.

3.2 Software Interfaces

Considering the scale of this project, there are multiple components to consider, and a lot of elements are subject to change. For the database, we're most likely using MySQL 8.0 for the construction process. This ensures that all of the pantry-related data is stored and managed properly. The OS will most likely be Linux-based considering how uniform the structure is. The main purpose would be to host the web/database servers, and background task schedulers. The main point of the web servers would be to handle the HTTP requests from the users and link them back to the code. The backend Applications would handle the business logic layers which process the pantry actions and interact with the database. Considering how we would still implement the notification system, the main action this service provides consists of sending emails that alert the

users about any stock updates. The implementation process consists of experimenting with various types of code and formatting to ensure that this process flows smoothly.

3.3 Communications Interfaces

One of the requirements that we'll use with this product is a modern browser that has HTTP support, with the main goal of interacting with the code and the encryption. The email's main functionality would be to notify any user about restocking when the items are close to their expiration or low. We can also have standards which aim to uniform the whole product (like accepting HTTPS, SMTP, HTTP, APIs, and standards along those lines). All web communications are encrypted and session data should be protected with some form of cookies. The messaging format should include the subject ([Pantry Alert] item(s) Expiring Soon), a greeting (Hello [User],), and a basic interface that shows all of the items and their expiration dates.

4. System Features

This section details the major services and functionalities provided by the Pantry Inventory Management System. Each feature is described in terms of its purpose, priority, user interactions (stimulus/response sequences), and specific functional requirements.

4.1 Private and Protected Inventory

4.1.1 Description and Priority

This feature will provide the user the authentication process, that will protect both staff and general users. Making sure the access to the pantry inventory is private and protected from unauthorized access.
***Priority:** HIGH*

4.1.2 Stimulus/Response Sequences

***User Action:** User navigates to the login/signup page.*
***System Response:** System displays the login and signup forms.*

***User Action:** Enter a valid signup information. (Email and password.)*
***System Response:** System creates a new account with the provided information and logs the user in.*

***User Action:** User enters valid login information.*
***System Response:** System authenticates the user and grants access to their personalized pantry.*

***User Action:** Enter an invalid login information.*
***System Response:** System displays a message informing that the information entered is incorrect ("Invalid email or password").*

4.1.3 Functional Requirements

REQ-1: The system will allow users to register for a new account by providing a unique email and password.
REQ-2: The system will allow users to log in to their account using their registered email and password.
REQ-3: The system will require users to create a password with a minimum length and a combination of uppercase letters, lowercase letters, and numbers.

4.2 Item Management System

4.2.1 Description and Priority

This feature allows users to manage their pantry items. They can add new items, update existing ones, and remove items. This includes details like names, quantities, and expiration dates.
Priority: *HIGH*

4.2.2 Stimulus/Response Sequences

User Action: *User logs into their account.*
System Response: *System displays the user's pantry inventory.*

User Action: *User selects to add a new item.*
System Response: *System displays a form to enter item details.*

User Action: *User enters item details (name, quantity, expiration date) and saves.*
System Response: *System adds the new item to the pantry and updates the display.*

User Action: *User selects an item to update.*
System Response: *System displays the item's details in an editable form.*

User Action: *User modifies item details and saves.*
System Response: *System updates the item's information and refreshes the display.*

User Action: *User selects an item to remove.*
System Response: *System prompts for confirmation.*

User Action: *User confirms removal.*
System Response: *System deletes the item from the pantry.*

4.2.3 Functional Requirements

REQ-1: The system will allow authenticated users to add new pantry items with name, quantity, and expiration date.
REQ-2: The system will allow authenticated users to update details of existing pantry items.
REQ-3: The system will allow authenticated users to remove pantry items.

4.3 Pantry Search

4.3.1 Description and Priority

This feature lets users search for items in their pantry based on criteria like quantity, name, price, and expiration date. It helps users quickly find specific items.
Priority: *MEDIUM*

4.3.2 Stimulus/Response Sequences

User Action: *User views their pantry inventory.*
System Response: *System displays all pantry items.*

User Action: *User enters a search query (item name, quantity, expiration date).*
System Response: *System filters and displays matching pantry items.*

User Action: *User clears the search query.*
System Response: *System displays all pantry items again.*

4.3.3 Functional Requirements

REQ-1: The system will provide a search function for pantry items.
REQ-2: Users will be able to search for items by name.
REQ-3: Users will be able to search for items by quantity.
REQ-4: Users will be able to search for items by expiration date.

4.4 Shopping List Creation

4.4.1 Description and Priority

This feature allows users to create and manage shopping lists. Users can sort items based on low stock or expiring items. They can also mark items as bought.
Priority: *MEDIUM*

4.4.2 Stimulus/Response Sequences

User Action: *User navigates to the shopping list section.*
System Response: *System displays existing shopping lists and an option to create a new one.*

User Action: *User creates a new shopping list.*
System Response: *System creates and displays an empty shopping list.*

User Action: *User adds an item to a shopping list.*
System Response: *System adds the item to the list.*

User Action: *User selects to sort items on the shopping list.*
System Response: *System reorders items based on selected criteria (low stock, expiration date)*

User Action: *User marks a list as bought.*
System Response: *System updates the list status.*

4.4.3 Functional Requirements

REQ-1: The system will allow the user to create multiple shopping lists.
REQ-2: The system will allow the user to add items to a shopping list.
REQ-3: The system will allow the user to mark a list as "purchased".
REQ-4: The system will allow users to sort shopping list items by criteria such as low stock or nearing expiration.

4.5 Remainder of Expiration Date

4.5.1 Description and Priority

This feature notifies users when their pantry items are nearing their expiration date. Notifications will be sent by email or SMS.
Priority: *HIGH*

4.5.2 Stimulus/Response Sequences

System Action: *System automatically checks for expiring items.*
System Response: *System triggers a notification process if items are nearing expiration.*

User Action: *User views their pantry inventory.*
System Response: *Expiring items are visually highlighted in the display.*

4.5.3 Functional Requirements

REQ-1: The system will automatically identify pantry items within a configurable expiration threshold.
REQ-2: The system will visually highlight expiring items in the pantry display.
REQ-3: The system will send notifications to users about expiring items via email or SMS.
REQ-4: Notifications will clearly state the expiring item(s) and their dates.

4.6 Basic Security Measures

4.6.1 Description and Priority

This feature ensures the integrity and protection of user data through input validation, role-based access control, and secure session management.
Priority: *HIGH*

4.6.2 Stimulus/Response Sequences

User Action: *User submits any data to the server (login, item details).*
System Response: *System sanitizes the input before processing.*

User Action: *User attempts to read or edit another user's pantry.*
System Response: *System denies access, as reading and editing pantries are restricted to the owner.*

User Action: *User successfully logs in.*
System Response: *System establishes a secure session using Cookies-Based Authentication or token authentication (JWT), keeping the user logged in during their session.*

4.6.3 Functional Requirements

REQ-1: All input into the server must be sanitized before being processed to prevent vulnerabilities.
REQ-2: Role-based access control will be implemented, ensuring that reading and editing of pantries are only permitted to the respective users.
REQ-3: Once logged in, users must remain logged in securely during their session, utilizing either Cookies-Based Authentication or token authentication through JWT

5. Other Nonfunctional Requirements

5.1 Performance Requirements

P.R-1 The system shall be responsive (< 1000 ms) to all queries with the assumption that the pantry holds no more than 500 items.

P.R-2 The system shall be responsive (< 1000 ms) under load with the assumption that there will be no more than 10 concurrent users.

5.2 Safety Requirements

Saf.R-1 The system shall track the expiration dates of all products in the pantry and dispose of them automatically.

5.3 Security Requirements

Sec.R-1 The software shall enforce strong authentication requiring high entropy passwords

Sec.R-2 All passwords must be hashed by either bcrypt, Argon2, scrypt, SHA-512 or PBKDF2 with a salt to prevent any rainbow/dictionary attacks.

Sec.R-3 All user input must be sanitized before being sent/processed by the server to prevent XSS or SQL injection attacks

Sec.R-4 User authentication must use cookies-based authentication (needs csrf tokens), or token authentication through (JWT)

5.4 Software Quality Attributes

Accessibility: The frontend shall be primarily designed for desktop web browsers and shall follow HTML 5 best practice for accessibility.

Availability: The application shall be accessible on demand via startup script.

Data Integrity: The database shall follow data storage best practices for data integrity. More in sec. 6.1

6. Other Requirements

6.1 Database Requirements

Db.R-1 The system must use a relational database (e.g., PostgreSQL) to ensure data consistency.

Db.R-2 All personally identifiable information (PII) must be encrypted at rest and in transit.

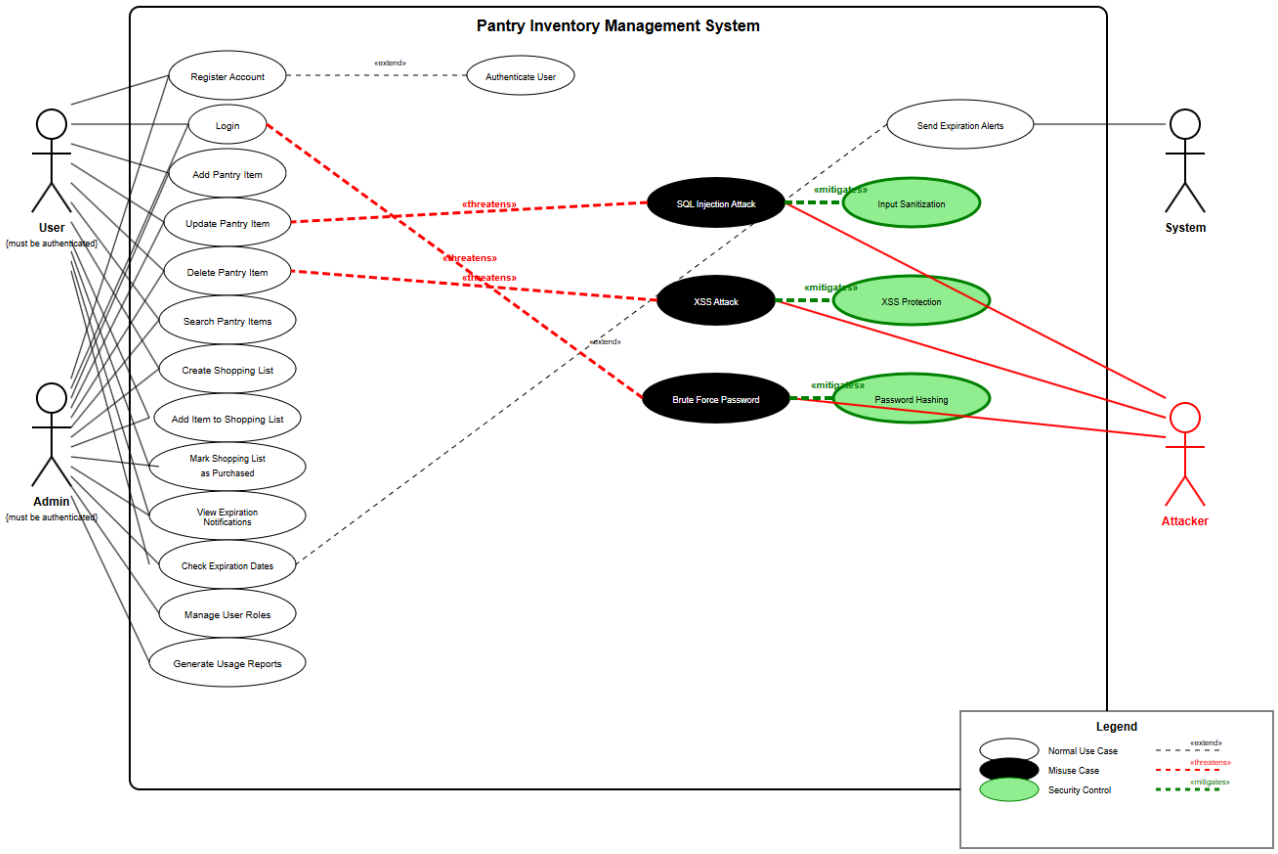
Db.R-3 The database should support daily automated backups with a 7-day retention period.

Db.R-4 The system must maintain referential integrity between related tables using foreign key constraints.

7. Use Case Model

7.1 Use Case Diagram

For clarification, users/admins **must** be authenticated for any and all use cases shown in the following diagram.



7.2 Use Case Descriptions

Use Case Number	UC-001		
Use Case Name	Register Account		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is not logged in		
Successful Post Condition	User is registered and logged into a new account		
Actors	User, Admin		
Priority	High		
Related Use Cases	Login, Logout		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click register	SR1	Return registration page
UA2	Enter valid form information and submit	SR2	Register account and login user
Alternative I: Invalid information			
User Action		**System Response**	
A1.UA1	Click register	A1.SR1	Return registration page
A1.UA2	Enter in-valid form information and submit	A1.SR2	Display error message and do not proceed

Use Case Number	UC-002

Use Case Name	Login		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is registered but not logged in		
Successful Post Condition	User is registered and logged into existing account		
Actors	User, Admin		
Priority	High		
Related Use Cases	Register, Logout		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click login	SR1	Return login page
UA2	Enter valid form information and submit	SR2	Successfully authenticate user
Alternative I: Non-existent Account			
User Action		**System Response**	
A1.UA1	Click login	A1.SR1	Return login page
A1.UA2	Enter non-existent user	A1.SR2	Inform user that the account does not exist
Alternative 2: Incorrect Password			
User Action		**System Response**	
A2.UA1	Click login	A2.SR1	Return login page
A2.UA2	Enter wrong password	A2.SR2	Inform user that the password is incorrect

Use Case Number	UC-003		
Use Case Name	Logout		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is currently logged in		
Successful Post Condition	User is logged out		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Register, Login		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click logout	SR1	Ask the user to confirm
UA2	Click Confirm	SR2	Successfully logout user
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click logout	A1.SR1	Ask the user to confirm
A1.UA2	Click Cancel	A1.SR2	Cancel logout process

Use Case Number	UC-004
Use Case Name	Add item to pantry
Author/Source	Andrew Feran
Date of Creation	6/1/2025
Precondition(s)	User is logged in and viewing the inventory page
Successful Post Condition	Item is in pantry or item quantity increment accordingly

Actors		User, Admin	
Priority		High	
Related Use Cases		Update Pantry Item, Delete Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click add item	SR1	Display a form in a modal
UA2	Enter valid form information and submit	SR2	Successfully add item to pantry
Alternative I: Invalid item			
User Action		**System Response**	
A1.UA1	Click add item	A1.SR1	Display a form in a modal
A1.UA2	Enter in-valid information	A1.SR2	Inform user that the item couldn't be added

Use Case Number		UC-005	
Use Case Name		Update item in pantry	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is logged in and viewing the inventory page	
Successful Post Condition		Item details are updated according to the user's request	
Actors		User, Admin	
Priority		High	
Related Use Cases		Add Pantry Item, Delete Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click update item	SR1	Display a form in a modal
UA2	Enter valid form information and submit	SR2	Successfully update item in pantry
Alternative I: Invalid information			
User Action		**System Response**	
A1.UA1	Click update item	A1.SR1	Display a form in a modal
A1.UA2	Enter in-valid information	A1.SR2	Inform user that the item couldn't be updated

Use Case Number		UC-006	
Use Case Name		Delete item in pantry	
Author/Source		Andrew Feran	
Date of Creation		6/1/2025	
Precondition(s)		User is logged in and viewing the inventory page	
Successful Post Condition		Item is deleted or count is decremented accordingly	
Actors		User, Admin	
Priority		High	
Related Use Cases		Add Pantry Item, Update Pantry Item, Search Pantry Item	
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click delete item	SR1	Display a confirmation alert
UA2	Confirm	SR2	Successfully remove item from pantry

Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click delete item	A1.SR1	Display a confirmation alert
A1.UA2	Cancel	A1.SR2	Do not remove the item from pantry

Use Case Number	UC-007		
Use Case Name	Search items in pantry		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the inventory page		
Successful Post Condition	Relevant items based on criteria are returned		
Actors	User, Admin		
Priority	High		
Related Use Cases	Add Pantry Item, Update Pantry Item, Delete Pantry Item		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Enter information to the search form	SR1	Do nothing
UA2	Click search	SR2	Query the database and return relevant items
Alternative I: Search All			
User Action		**System Response**	
A1.UA1	Enter no information to the search form	A1.SR1	Do nothing
A1.UA2	Click search	A1.SR2	Query database and return all items
Alternative 2: Item(s) not found			
User Action		**System Response**	
A2.UA1	Enter information to the search form	A2.SR1	Do nothing
A2.UA2	Click search	A2.SR2	Query the database and respond “no items…”
Alternative 3: Search All			
User Action		**System Response**	
A3.UA1	Enter no information to the search form	A3.SR1	Do nothing
A3.UA2	Click search	A3.SR2	Query the database and respond “no items…”

Use Case Number	UC-008		
Use Case Name	Create Shopping List		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the shopping list page		
Successful Post Condition	Shopping list is created per user’s request		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Add item to list, Mark list as purchased		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click create list	SR1	Display a form in a modal
UA2	Enter valid form information and submit	SR2	Successfully add list
Alternative I: Invalid information			
User Action		**System Response**	

A1.UA1	Click create list	A1.SR1	Display a form in a modal
A1.UA2	Enter in-valid information	A1.SR2	Inform user that the list couldn't be made

Use Case Number	UC-009		
Use Case Name	Add item to list		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the inventory page		
Successful Post Condition	Shopping list is updated per user's request		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Create shopping list, Mark list as purchased		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click add to list	SR1	Display a confirmation
UA2	Click confirm	SR2	Successfully add item to list
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click add to list	A1.SR1	Display a confirmation
A1.UA2	Click Cancel	A1.SR2	Do nothing

Use Case Number	UC-010		
Use Case Name	Mark list as purchased		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing a shopping list		
Successful Post Condition	Shopping list deleted and inventory updated accordingly		
Actors	User, Admin		
Priority	Medium		
Related Use Cases	Add item to list, Create shopping list		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click "mark as purchased"	SR1	Display a confirmation
UA2	Click confirm	SR2	Delete list, update inventory accordingly
Alternative I: Cancel			
User Action		**System Response**	
A1.UA1	Click "mark as purchased"	A1.SR1	Display a confirmation
A1.UA2	Click Cancel	A1.SR2	Do nothing

Use Case Number	UC-011
Use Case Name	View expiration notifications
Author/Source	Andrew Feran

Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the notifications tab		
Successful Post Condition	User can view ANY and ALL expired items in the pantry		
Actors	User, Admin		
Priority	Urgent		
Related Use Cases	Check expiration dates, Send expiration alerts		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Click notifications tab	SR1	Show notifications
Alternative I: No notifications to display			
User Action		**System Response**	
A1.UA1	Click notifications tab	A1.SR1	Show message “No expiration notifications”

Use Case Number	UC-012		
Use Case Name	Check expiration dates		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is logged in and viewing the inventory page		
Successful Post Condition	User can determine the expiration dates of products		
Actors	User, Admin		
Priority	Urgent		
Related Use Cases	View expiration notifications, Send expiration alerts		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	User clicks and item’s information	SR1	Modal pops up
UA2	User reads information and hits close	SR2	Modal closes

Use Case Number	UC-013		
Use Case Name	Manage user roles		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is authenticated with admin permissions		
Successful Post Condition	Administrator manages roles of other users		
Actors	Admin		
Priority	High		
Related Use Cases			
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Navigate to admin area	SR1	Display admin area
UA2	Click to users page	SR2	Display users page
UA3	Click user	SR3	Display modal
UA4	Enter user roles and confirm	SR4	Update user roles and close modal

Use Case Number	UC-014		
Use Case Name	Generate usage reports		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	User is authenticated with admin permissions		
Successful Post Condition	Administrator can generate usage reports		
Actors	Admin		
Priority	Low		
Related Use Cases			
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Navigate to admin area	SR1	Display admin area
UA2	Click to usage page	SR2	Display usage page
UA3	Click to export usage to csv	SR3	Compile usage to CSV and upload

Use Case Number	UC-015		
Use Case Name	Send expiration alerts		
Author/Source	Andrew Feran		
Date of Creation	6/1/2025		
Precondition(s)	There are expiration alerts that need to be sent		
Successful Post Condition	SMTP receipt is confirmed on the alerts		
Actors	User		
Priority	Urgent		
Related Use Cases	Check expiration dates, View expiration notifications		
Flow of Events			
Main Flow			
User Action		**System Response**	
UA1	Has expired items in pantry	SR1	Send email alert
UA2	SMTP server responds with received	SR2	Success!
Alternative I: SMTP not received			
User Action		**System Response**	
A1.UA1	Has expired items in pantry	A1.SR1	Send email alert
A1.UA2	SMTP server responds with failed	A1.SR2	Log issue for Sys. Admins

Appendix A: Glossary

Authentication & Security Terms:

- **JSON Web Token (JWT):** Standard for securely transmitting information between parties as tokens
- **bcrypt:** Password hashing function designed to be slow and resistant to brute-force attacks
- **Argon2:** Modern password hashing function recommended for secure password storage

- **scrypt:** Password-based key derivation function designed to be memory-hard
- **Password-Based Key Derivation Function 2 (PBKDF2):** Password-Based Key Derivation Function used for password hashing
- **Secure Hash Algorithm 512 (SHA-512):** Cryptographic hash function producing 512-bit hash values
- **Salt:** Random data added to passwords before hashing to prevent rainbow table attacks
- **Cross-Site Request Forgery (CSRF) tokens:** Security tokens used to prevent Cross-Site Request Forgery attacks
- **Session-based authentication:** a stateful approach where a server maintains a record of a user's authenticated state for a specific period. Needs CSRF tokens to be secure
- **Stateless vs Stateful:** Stateless systems treat each request as a new, independent event, while stateful systems maintain information about past interactions to provide a more personalized or continuous experience.
- **Session management:** Process of maintaining user login state during application use
- **Rainbow attacks:** Cryptographic attack using precomputed tables of hash values to crack passwords
- **Dictionary attacks:** Password cracking method using lists of common passwords
- **Salt:** random data added to a password before it is hashed and stored protects against rainbow attacks
- **TLS (Transport Layer Security)**: a cryptographic protocol that provides privacy and data integrity for communications over the internet. It secures Protocols like https previously called SSL newest version is 1.3
- **Hashing vs Encryption:** Encryption protects data confidentiality by making it unreadable to unauthorized users, while hashing ensures data integrity and can be used for password storage. Hashing is irreversible, meaning it's computationally infeasible to recover the original data from the hash, whereas encryption is reversible with a key.
- **Sanitation:** process of cleaning and modifying user input to remove or neutralize any potentially harmful characters or code. Protects against SQL injection and XSS
- **Two-factor authentication (2FA)**: security measure that requires users to provide two separate forms of identification to verify their identity when logging into an account protects against dictionary attacks

Database Terms:

- **MySQL:** is an open-source relational database management system
- **PostgreSQL:** is an advanced open-source relational database system
- **Structured Query Language (SQL):** is a programming language for managing relational databases
- **Primary key:** a column (or a group of columns) that uniquely identifies each row in a table
- **Foreign key constraints:** Database rules that maintain referential integrity between tables
- **Referential integrity:** Database property ensuring relationships between tables remain consistent
- **Personally Identifiable Information (PII):** Data that can identify a specific individual
- **SQL injection:** an attack in which the attacker injects malicious SQL code into an application, allowing the attacker to view or modify a database

Web Technologies:

- **HyperText Transfer Protocol/HyperText Transfer Protocol Secure (HTTP/HTTPS):** are common communication protocols for web applications (secure version uses encryption)

- **HyperText Markup Language 5 (HTML5):** is the latest version of HyperText Markup Language for web content
- **Cascading Style Sheets 3 (CSS3):** Latest version of Cascading Style Sheets for web styling
- **JavaScript (ES6):** is a programming language for web interactivity (ES6 is ECMAScript 2015)
- **Apache Server:** is an open-source web server software
- **Flask:** Python web framework for building web applications
- **Application Programming Interface (API):** interface for software component communication
- **Simple Mail Transfer Protocol (SMTP):** is a common protocol for sending emails
- **POP3 (Post Office Protocol 3):** downloads emails to your device and then typically deletes them from the server
- **IMAP (Internet Message Access Protocol):** keeps emails on the server and syncs them across multiple devices
- **XSS (Cross-Site Scripting):** type of attack which allows attackers to inject malicious scripts into web pages, and execute them on other user's browser's unknowingly

System Terms:

- **Ubuntu Server:** Linux-based operating system for servers
- **Python 3:** Programming language used for the backend application